

Description “ Commit 6 ”

- มีการเขียนการทำงานโค้ดเพิ่มเติม ใน function ต่อไปนี้

- `Board.clear_lines()` - ตรวจสอบแถวและคอลัมน์ที่เต็ม แล้วล้างให้กลับเป็นช่องว่าง พร้อมคำนวณคะแนนที่ได้
- `is_hand_empty()` - ตรวจสอบว่าชิ้นส่วนในมือถูกใช้หมดทุกช่องแล้วหรือยัง
- `check_game_over()` - ตรวจสอบว่าเกมจบหรือยัง โดยลองวางชิ้นส่วนในมือลงทุกช่องของกระดาน
- `mouseReleased()` - เมื่อปล่อยเมาส์ ตรวจสอบว่าวางได้ไหม ถ้าวางได้ให้วางลงกระดาน คัดคะแนนและเตรียมตาถัดไป ถ้าวางไม่ได้ให้ชิ้นส่วนกลับที่เดิม
- `draw()` - เพิ่มการแสดงผลคะแนน สถานะเกม ชิ้นส่วนที่กำลังลาก และข้อความเมื่อเกมจบ

- มีการแก้ไขเล็กน้อยใน `Board.draw()`, `Board.place()` และ `Piece.reset_pos()` (ดูหัวข้อสุดท้าย)

Board.clear_lines(self):

ตรวจสอบทุกแถวและทุกคอลัมน์ว่าเต็มหรือไม่ (ไม่มีช่องใดเป็น 0) แล้วล้างแถว/คอลัมน์ที่เต็มให้กลับเป็น 0

คำนวณคะแนนที่ได้จากการล้าง

1. **เตรียมลิสต์ว่างเก็บผลลัพธ์** สร้าง `rows_to_clear` และ `cols_to_clear` เริ่มต้นเป็นลิสต์ว่างไว้เก็บ “เลขแถว” และ “เลขคอลัมน์” ที่เต็ม
2. **ตรวจสอบแถวที่เต็ม** ใช้ลูป 2 ชั้น ลูปนอกเลือกทีละแถว (`r`) ลูปในไล่ดูทีละช่องในแถวนั้น (`c`) ถ้าช่องไม่ว่าง (`grid[r][c] != 0`) ให้เพิ่มตัวนับ `filled` ทีละ 1 เมื่อจบแถว ถ้า `filled == self.size` (ครบ 8 ช่อง) แปลว่าแถวเต็มทั้งหมด จึง `append` เลขแถวนั้น ๆ ลง `rows_to_clear` โดยต้องตั้ง `filled = 0` ใหม่ทุกครั้งที่เราเริ่มแถวใหม่
3. **ตรวจสอบคอลัมน์ที่เต็ม** ทำแบบเดียวกับข้อ 2 แต่ลูปนอกเลือกทีละคอลัมน์ และลูปในไล่ลงตามแนวตั้ง (`grid[r][c]` โดยให้ `r` เป็นตัวไล่ทีละแถวแทน) ถ้านับได้ครบ 8 ช่อง จึง `append` เลขคอลัมน์ลง `cols_to_clear`
4. **ล้างแถวที่เต็ม** วนตามเลขแถวใน `rows_to_clear` แล้วตั้งค่าทุกช่องของแถวนั้นเป็น 0
5. **ล้างคอลัมน์ที่เต็ม** วนตามเลขคอลัมน์ใน `cols_to_clear` แล้วตั้งค่าทุกช่องของคอลัมน์นั้นเป็น 0
6. **คืนคะแนน** `return (len(rows_to_clear) + len(cols_to_clear)) * 100` คือ (จำนวนแถวที่ล้าง + จำนวนคอลัมน์ที่ล้าง) $\times$ 100

*** เหตุผลที่ต้องตรวจสอบแถวและคอลัมน์ให้ครบก่อน แล้วค่อยล้าง เพราะถ้าล้างทันทีที่เจอแถวเต็ม

ช่องในแถวนั้นจะกลายเป็น 0 ทำให้ช่องที่คอลัมน์ติดกับแถวนั้นกลายเป็นไม่เต็มและไม่ถูกนับ การแยกเป็นตรวจสอบให้ครบก่อนล้างทีหลัง จึงทำให้กรณีแถวและคอลัมน์เต็มพร้อมกัน (ตัดกันเป็นรูปกากบาท) ได้คะแนนครบทั้งสอง

is_hand_empty():

ตรวจสอบว่าชิ้นส่วนทุกช่องในมือ (hand) ถูกใช้ไปหมดแล้วหรือยัง คืนค่า True ถ้าทุกช่องว่าง และ False ถ้ายังเหลืออย่างน้อย 1 ชิ้น

1. วนดูในลิสต์ hand ทีละช่อง ไล่ตั้งแต่ช่องที่ 0 ถึงช่องสุดท้าย
2. เจอชิ้นส่วนที่ยังเหลือ ถ้าช่องนั้นไม่ใช่ 0 (ยังมีชิ้นส่วนอยู่) ให้ return False ทันที ไม่ต้องตรวจช่องที่เหลือ
3. ไม่เจอเลย เมื่อวนครบทุกช่องแล้วไม่มีชิ้นส่วนเหลือ จึง return True

ฟังก์ชันนี้ถูกเรียกใน mouseReleased() เพื่อตัดสินใจว่าต้องเรียก spawn_hand() สุ่มชุดใหม่ 3 ชิ้นหรือไม่

check_game_over():

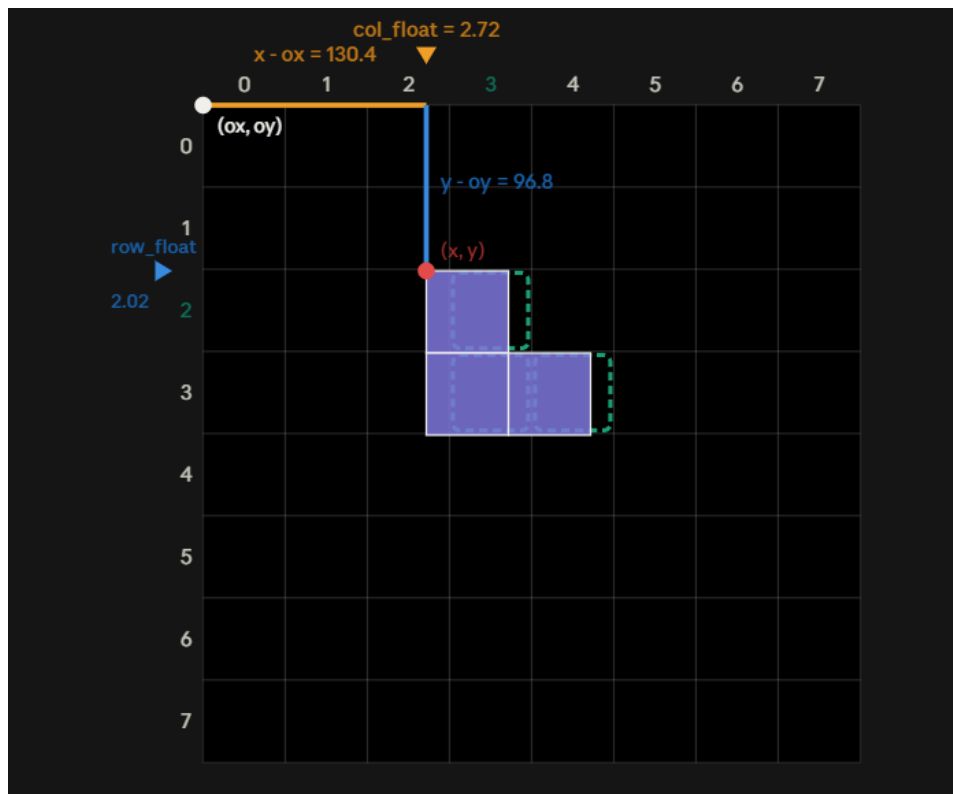
ตรวจสอบว่าเกมจบหรือยัง โดยลองวางชิ้นส่วนในมือลงทุกช่องของกระดาน ถ้าไม่มีชิ้นใดวางได้เลยแม้แต่ช่องเดียว ถือว่าเกมจบ

1. รวบรวมชิ้นส่วนที่รูปทรงไม่ซ้ำกันลงในลิสต์ unique_pieces วนดูในลิสต์ hand ซ้ำช่องที่เป็น 0 แล้วเทียบของชิ้นนั้นกับชิ้นที่เก็บไว้ในลิสต์ unique_pieces ถ้ารูปทรงยังไม่ซ้ำจึงเพิ่มเข้าลิสต์แล้วนำไปทดลองวาง เหตุผลที่เก็บเฉพาะรูปทรงไม่ซ้ำกัน เพราะถ้ามีรูปทรงเดียวกันหลายชิ้น ไม่จำเป็นต้องลองวางซ้ำทุกชิ้น ลองแค่ชิ้นเดียวก็เพียงพอ เพราะผลลัพธ์ที่ได้เหมือนกัน
 2. สมมติว่าเกมจบไว้ก่อน ตั้ง game_over = True แล้วค่อยเปลี่ยนเป็น False เมื่อเจอที่ว่าง
 3. ลองวางทุกชิ้นลงทุกช่อง ใช้ลูป 3 ชั้น คือ เลือกชิ้นส่วน (k) เลือกแถว (r) และเลือกคอลัมน์ (c) แล้วเรียก board.can_place(unique_pieces[k], r, c) ถ้าวางได้ ตั้งค่า game_over = False และหยุดทุกลูปทันทีที่เจอที่ว่าง
 4. คืนค่า return game_over โดย True = เกมจบ (ไม่มีที่ว่างเหลือ) และ False = ยังเล่นได้
- ** ฟังก์ชันนี้ต้องถูกเรียกหลังจากเติมชิ้นส่วนในมือใหม่แล้ว เพื่อตรวจสอบก่อนวางอีกครั้งว่ายังเล่นได้หรือไม่

mouseReleased():

ทำงานเมื่อผู้เล่นปล่อยเมาส์ มีหน้าที่ตรวจสอบว่าปล่อยชิ้นส่วนลงตำแหน่งที่ว่างได้หรือไม่ ถ้าวางได้ให้วางลงกระดานและเตรียมตาถัดไป ถ้าวางไม่ได้ให้ชิ้นส่วนกลับที่เดิม

1. เช็ควากำลังถือชิ้นส่วนอยู่ไหม ถ้า selected_piece == None (ปล่อยเมาส์โดยไม่ได้ถืออะไร) ให้ return ออกทันที
2. แปลงตำแหน่งพิกเซลเป็นตำแหน่งช่องบนกระดาน
นำตำแหน่งมุมซ้ายบนของชิ้นส่วนลบด้วยจุดเริ่มกระดาน (board.ox, board.oy) แล้วหารด้วยขนาดช่อง ได้ col_float และ row_float จากนั้นปัดเป็นช่องที่ใกล้ที่สุดด้วย int(round(...)) ได้ target_c (จากแกน x) และ target_r (จากแกน y)



```
x = 188.4, y = 156.8 ox = 58, oy = 60, cell_size = 48
```

ขั้น 1 ลบจุดเริ่มกระดาน: $(188.4 - 58, 156.8 - 60) = (130.4, 96.8)$

ขั้น 2 หาขนาดช่อง: $col_float = 130.4 / 48 = 2.72$, $row_float = 96.8 / 48 = 2.02$

ขั้น 3 บัดเศษ: $target_c = \text{int}(\text{round}(2.72)) = 3$, $target_r = \text{int}(\text{round}(2.02)) = 2$ (กรอบเขียว = ช่องที่จะวาง)

3. ตรวจสอบว่าวางได้ไหม เรียก `board.can_place(selected_piece, target_r, target_c)`
4. กรณีวางได้ ทำตามลำดับนี้
 - เรียก `board.place()` เขียนสีลงกระดาน
 - คำนวณคะแนน $\text{gained} = (\text{จำนวน block} \times 10) + \text{board.clear_lines}()$ แล้วบวกเข้า `score`
 - เอาชิ้นที่ใช้แล้วออกจากมือ `hand[selected_index] = 0`
 - ถ้า `is\_hand\_empty()` เป็นจริง (ใช้ครบ 3 ชิ้น) เรียก `spawn\_hand()` สุ่มชุดใหม่
 - ตรวจสอบเกมจบ `game\_over = check\_game\_over()`
5. กรณีวางไม่ได้ เรียก `selected_piece.reset\_pos()` ให้ชิ้นส่วนกลับไปอยู่ที่เดิมในมือ
6. เคลียร์การเลือก ตั้ง `selected_piece = None` และ `selected_index = -1` เพื่อล้างสถานะจากกำลังถือชิ้นส่วนอยู่ ให้กลับเป็น "ไม่ได้ถืออะไร" หลังจบการลากหนึ่งรอบ

draw():

วาดภาพ และข้อความ ดังนี้

1. **แถบด้านบน** แสดงคะแนน “Score: ...” ทางซ้าย และสถานะเกมทางขวา ถ้า `game_over` เป็นจริงแสดง “Status: YOU LOSE” สีแดง ไม่เช่นนั้นแสดง “Status: Playing” สีเขียว
2. **วาดชิ้นส่วนในมือ** ข้ามช่องที่ว่าง (0) และข้ามชิ้นที่กำลังลากอยู่ (`is_dragging` เป็นจริง) เพราะชิ้นที่กำลังลากจะถูกวาดแยกในข้อถัดไป ไม่ให้ถูกวาดซ้ำสองรอบ
3. **วาดชิ้นที่กำลังลากไว้บนสุด** ถ้า `selected_piece != None` ให้เรียก `draw()` หลังจากวาดกระดานและชิ้นอื่นครบแล้ว เพราะลำดับมีผลสำคัญ สิ่งทีวาดทีหลังจะทับสิ่งที่วาดก่อน ต้องให้ชิ้นที่กำลังลากอยู่บนสุด ดังนั้นต้องวาดทีหลัง
4. **เมื่อเกมจบ** ถ้า `game_over` เป็นจริง แสดงข้อความ “YOU LOSE” ขนาดใหญ่กลางจอ และ “Click to restart” ซึ่งตรงกับ `mousePressed()` ที่เมื่อคลิกตอนเกมจบจะเรียก `setup()` เริ่มเกมใหม่

การแก้ไขเล็กน้อยจาก Commit 5

- `Board.draw()` ใช้สีกรอบเป็นสีดำกับทุกช่อง (เดิมช่องที่มี block ใช้สีกรอบเท่าสีพื้น ทำให้มองไม่เห็นเส้นแบ่งช่อง)
- `Board.place()` เขียนค่าลงกระดานด้วยตัวแปร `color_value` ที่คำนวณไว้แล้ว (เดิมคำนวณ `piece.color_idx + 1` เข้าในบรรทัดเขียน)
- `Piece.reset_pos()` แก้เป็น `self.anchor_x` และ `self.anchor_y` (เดิมไม่มี `self`. จึงเกิด `NameError` ตอนวางไม่ได้)
- `selected_index` เพิ่มคอมเมนต์อธิบายว่าค่า -1 คือค่าเริ่มต้นที่แปลว่ายังไม่ได้เลือกชิ้นไหน

Description “Commit 7”

- มีการเขียนการทำงานโค้ดเพิ่มเติม ใน function ต่อไปนี้

- `mouseReleased()` - เพิ่มระบบ combo streak และข้อความ CLEAR +N เมื่อล้างแถว/คอลัมน์ได้
- `draw_ghost_preview()` และ `draw_outline()` - แสดงกรอบบอกช่องที่ชิ้นส่วนจะลงขณะลาก
- `draw_hud()`, `draw_clear_msg()`, `draw_status_notice()` - แสดงแถบคะแนน สถานะ streak ข้อความ CLEAR และข้อความแจ้งผล save/load
- `keyPressed()`, `save_game()`, `load_game()` - บันทึกและโหลดเกมด้วยปุ่ม S และ L
- `make_piece()`, `my_split()` - ฟังก์ชันช่วยสร้างชิ้นส่วน และตัดข้อความที่อ่านจากไฟล์ save
- `draw()` - แยกการวาดออกเป็นฟังก์ชันย่อย 6 ฟังก์ชัน ทำให้ `draw()` สั้นลง

- มีการแก้ไขเล็กน้อยใน Piece.__init__(), spawn_hand(), setup() และ mousePressed()
- ตัวแปรที่เพิ่มใหม่: combo_streak, status_notice, status_timer, clear_msg, clear_timer (ใช้กับระบบ streak และแสดงผลข้อความ) , SAVE_FILE และ attribute Piece.template_idx (เพื่อใช้กับระบบ save)

mouseReleased():

ใน commit 6 คิดคะแนนเป็นก้อนเดียว (จำนวน block $\times$ 10 + คะแนนจาก clear_lines() ใน commit 7 แยกคะแนนเป็นส่วนๆ และเพิ่มระบบ combo streak กับข้อความ CLEAR ในกรณีที่เคลียร์เส้นตรงได้

1. แยกคะแนนเป็น 2 ส่วน piece_points = len(blocks) $\times$ 10 คือคะแนนจากการวาง และ line_points = board.clear_lines() คือคะแนนจากแถว/คอลัมน์เส้นตรงที่ล้างได้
2. กรณีล้างได้ (line_points > 0) เพิ่ม combo_streak ที่ละ 1 แล้วคิดคะแนนโบนัส
 $combo_bonus = (combo_streak - 1) \times 50$ ดังนั้นการล้างครั้งแรก combo_streak = 1 ยังไม่มีโบนัส ครั้งที่ 2 ได้ 50 ครั้งที่ 3 ได้ 100 ตามลำดับ จากนั้นตั้ง clear_msg = "CLEAR +" ตามด้วย line_points และตั้ง clear_timer = 60 เพื่อให้ข้อความแสดงประมาณ 1 วินาที
3. กรณีล้างไม่ได้ ตั้ง combo_streak = 0 และ combo_bonus = 0 คือถ้าวางแล้วไม่ล้างอะไรเลย streak จะกลับเป็นศูนย์
4. รวมคะแนน score = score + piece_points + line_points + combo_bonus
5. ส่วนที่เหลือเหมือน commit 6 เอาชิ้นที่ใช้แล้วออกจากมือ เติมมือใหม่ถ้าหมด และตรวจเกมจบ
 ส่วนกรณีวางไม่ได้เรียก reset_pos() แล้วเคลียร์ selected_piece และ selected_index

ฟังก์ชันนี้ต้องประกาศ global เพิ่มอีก 3 ตัว คือ combo_streak, clear_msg และ clear_timer

**** ข้อความ CLEAR แสดงเฉพาะคะแนนจากแถว/คอลัมน์ที่ล้าง ไม่รวมโบนัส combo**

draw_clear_msg():

แสดงข้อความ "CLEAR +N" ที่มุมขวาบนของจอเมื่อผู้เล่นล้างแถวหรือคอลัมน์ได้ และหายไปเองหลังเวลาที่กำหนด

1. เช็ค clear_timer ทำงานเฉพาะเมื่อ clear_timer > 0 คือยังอยู่ในช่วงเวลาที่ต้องแสดงข้อความ
2. วาดข้อความ clear_msg ที่ตำแหน่ง (width - 120, 52)
3. นับถอยหลัง ลด clear_timer ลง 1 ทุกเฟรม เนื่องจาก draw() ทำงานประมาณ 60 ครั้งต่อวินาที ค่า 60 จึงเท่ากับประมาณ 1 วินาที เมื่อถึง 0 ข้อความหายไปเอง

draw_outline(x, y, size, color, weight):

วาดเฉพาะกรอบสี่เหลี่ยม ไม่มีสีพื้น ใช้วาด ghost preview เพื่อให้เห็นช่องและสีของกระดานที่อยู่ด้านล่าง

1. **ตั้งค่าเส้น** สีเส้นตาม color และความหนาตาม weight
2. **วาดเส้น 4 ด้าน** ด้วย line() คือขอบบน ขอบล่าง ขอบซ้าย และขอบขวา โดย x, y เป็นมุมซ้ายบน และ size เป็นความยาวด้าน

draw_ghost_preview():

ขณะผู้เล่นลากชิ้นส่วน แสดงกรอบบนกระดานเพื่อบอกล่วงหน้าว่าถ้าปล่อยตอนนี้ ชิ้นส่วนจะลงช่องไหน

1. **เช็คว่ากำลังลากอยู่ไหม** ทำงานเฉพาะเมื่อ selected_piece != None
2. **คำนวณช่องเป้าหมาย** ได้ g_c และ g_r จากตำแหน่งมุมซ้ายบนของชิ้นส่วน ด้วยสูตรเดียวกับ mouseReleased() คือ $\text{int}(\text{round}((x - \text{board.ox}) / \text{cell_size}))$ และ $\text{int}(\text{round}((y - \text{board.oy}) / \text{cell_size}))$ เพื่อแปลงตำแหน่ง pixel เป็นตำแหน่งบนช่องกระดาน
3. **ตรวจว่าวางได้ไหม** เรียก board.can_place(selected_piece, g_r, g_c) ถ้าวางไม่ได้จะไม่วาดอะไร ผู้เล่นจึงรู้ทันทีว่าตำแหน่งนั้นวางไม่ได้
4. **วาดกรอบที่ละ block** ถ้าวางได้ วนทุก block คำนวณมุมซ้ายบนบนจอของช่องนั้น
$$\text{gx} = \text{board.ox} + (\text{g_c} + \text{blocks}[i][0]) \times \text{cs}$$
$$\text{gy} = \text{board.oy} + (\text{g_r} + \text{blocks}[i][1]) \times \text{cs}$$
แล้วเรียก draw_outline(gx, gy, cs, g_color, 3) ด้วยสีเดียวกับชิ้นส่วน

*** ghost preview ถูกวาดหลัง board.draw() และก่อนวาดชิ้นส่วน

เพื่อให้กรอบอยู่บนกระดานแต่ไม่บังชิ้นที่กำลังลาก

draw_hud():

วาดข้อความแถบด้านบนของจอ ซึ่งใน commit 6 อยู่ใน draw() ใน commit 7 แยกออกมาเป็นฟังก์ชันของตัวเอง และเพิ่มข้อมูลบางส่วน

1. **คะแนน** แสดง “Score: ...” ที่มุมซ้ายบน
2. **ปุ่มลัด** แสดงข้อความตัวเล็ก “[S] Save | [L] Load” ใต้คะแนน บอกผู้เล่นว่ากดปุ่มใดเพื่อบันทึกหรือโหลดเกม
3. **สถานะเกม** ถ้า game_over เป็นจริงแสดง “Status: YOU LOSE” สีแดง ไม่เช่นนั้นแสดง “Status: Playing” สีเขียว

4. จำนวน **streak** ถ้า `combo_streak > 1` แสดง “Streak x N” สีส้ม ได้สถานะเกม

draw_hand(), draw_status_notice(), draw_game_over():

1. **draw_hand()** วาดชิ้นส่วนในมือ โดยข้ามช่องที่ว่าง (0) และข้ามชิ้นที่ `is_dragging` เป็นจริง จากนั้นวาดชิ้นที่กำลังลากไว้บนสุด (เหมือนส่วนที่อยู่ใน `draw()` ของ `commit 6`)
2. **draw_status_notice()** แสดงข้อความแจ้งผล `save/load (status_notice)` ได้กระดาน เมื่อ `status_timer > 0` โดยลด `status_timer` ลง 1 ทุกเฟรม ข้อความจึงหายไปเองหลังประมาณ 1 วินาที ใช้หลักการเดียวกับ **draw_clear_msg()**:
3. **draw_game_over()** ถ้า `game_over` เป็นจริง แสดง “YOU LOSE” ขนาดใหญ่กลางจอ และ “Click to restart” (เหมือนเดิมจาก `commit 6`)

draw():

ใน `commit 7` ฟังก์ชัน `draw()` ถูกแยกการวาดเป็นฟังก์ชันย่อย

เหลือเพียงล้างพื้นหลังแล้วเรียกฟังก์ชันย่อยตามลำดับ ทำให้อ่านง่ายและแก้ไขแต่ละส่วนได้สะดวก

1. **background()** ล้างพื้นหลังสีเทา
2. **draw_hud()** และ **draw_clear_msg()** วาดข้อความแถบด้านบนและข้อความ CLEAR
3. **board.draw()** วาดกระดาน
4. **draw_ghost_preview()** วาดกรอบ ghost บนกระดาน
5. **draw_hand()** วาดชิ้นส่วนในมือ และชิ้นที่ลากไว้บนสุด
6. **draw_status_notice()** และ **draw_game_over()** วาดข้อความแจ้งผล และข้อความเกมจบไว้ท้ายสุด

**** ลำดับการเรียกสำคัญ เพราะสิ่งที่วาดทีหลังจะทับสิ่งที่วาดก่อน**

keyPressed():

ถูกเรียกอัตโนมัติเมื่อผู้เล่นกดแป้นพิมพ์

1. กด **S** หรือ **s** เรียก `save_game()` เพื่อบันทึกเกม
2. กด **L** หรือ **l** เรียก `load_game()` เพื่อโหลดเกม

เช็คทั้งตัวพิมพ์เล็กและตัวพิมพ์ใหญ่ เพื่อให้ใช้ได้แม้เปิดโหมด Caps Lock อยู่

save_game():

บันทึกสถานะเกมปัจจุบันลงไฟล์ `savegame.txt` จำนวน 3 บรรทัด ได้แก่ กระดาน คะแนนกับ `streak` และชิ้นส่วนในมือ

1. เปิดไฟล์สำหรับเขียน ครอบด้วย try/except เพื่อป้องกันโปรแกรมหยุดทำงานถ้าเขียนไฟล์ไม่ได้
2. บรรทัดที่ 1: กระดาน สร้างข้อความด้วยการต่อ string ในรูป 2 ชั้น แต่ละแถวขึ้นต้นด้วย [และปิดด้วย] ค่าในแถวคั่นด้วย , และแต่ละแถวคั่นด้วย / โดยไม่ใส่ตัวคั่นหลังตัวสุดท้าย
3. บรรทัดที่ 2: คะแนน เขียน score และ combo_streak คั่นด้วย ,
4. บรรทัดที่ 3: ชิ้นส่วนในมือ แต่ละช่องเขียนเป็น ลำดับรูปทรง:ลำดับสี (template_idx:color_idx) หรือ EMPTY ถ้าช่องนั้นถูกใช้ไปแล้ว คั่นแต่ละช่องด้วย ;
5. ปิดไฟล์และแจ้งผล แสดง “Game Saved!” ถ้าเกิดข้อผิดพลาดแสดง “Save Failed!” แล้วตั้ง status_timer = 60 เพื่อบันทึกข้อมูลหลังข้อความหาย 60 วิ

my_split(text, sep):

ตัดข้อความตามตัวคั่น (sep) ที่ต้องการ โดยไล่อ่านทีละตัวอักษรด้วย while loop ใช้อ่านข้อมูลจากไฟล์ save

1. เตรียมตัวแปร parts เป็นลิสต์เก็บชิ้นข้อความที่ตัดได้ และ current เป็นข้อความที่กำลังสะสม
2. เจอตัวคั่น ถ้าตัวอักษรเท่ากับ sep ให้ append current ลง parts แล้วเริ่มสะสมชิ้นใหม่
3. ข้ามอักขระที่ไม่ใช่ข้อมูล ข้าม [] และตัวขึ้นบรรทัดใหม่ (\n, \r) เพื่อให้อ่านรูปแบบ [0,0,...] ได้
4. เก็บชิ้นสุดท้าย เมื่อวนจบ ชิ้นสุดท้ายไม่มีตัวคั่นปิดท้าย จึงต้อง append current อีกครั้งหลังลูป แล้ว return parts

load_game():

อ่านไฟล์ savegame.txt แล้วคืนสถานะเกมให้เหมือนตอนบันทึกครั้งล่าสุด ผู้เล่นจึงเล่นได้ทันที

โดยคะแนนนับต่อจากเดิม

1. เปิดไฟล์สำหรับอ่าน ถ้าเปิดไม่ได้ (ยังไม่เคย save) แสดง “No Save Found!” แล้ว return ออกทันที
2. ตรวจสอบจำนวนบรรทัด ต้องมีอย่างน้อย 3 บรรทัด ไม่เช่นนั้นแสดง “Corrupt Save!”
3. แปลงข้อมูลเก็บในตัวแปรชั่วคราวก่อน ได้แก่ new_grid, new_score, new_streak และ new_hand โดยบรรทัดที่ 1 ใช้ my_split(..., “/”) แยกแถว แล้ว my_split(..., “,”) แยกค่าแต่ละช่อง แปลงด้วย int() บรรทัดที่ 2 แยกคะแนนกับ streak และบรรทัดที่ 3 แยกช่องด้วย “;” แล้วแยกรูปทรงกับสีด้วย “:” จึงสร้างชิ้นส่วนด้วย make_piece()
4. ถ้าแปลงผิดพลาด แสดง “Corrupt Save!” แล้ว return เกมที่กำลังเล่นอยู่จะไม่ถูกเปลี่ยน เพราะยังไม่ได้เขียนทับค่าจริง
5. ถ้าถูกต้อง นำมาใช้จริง ใส่ค่าลง board.grid, score, combo_streak และ hand
6. ล้างการเลือก และตรวจเกมจบใหม่ ตั้ง selected_piece = None, selected_index = -1 และเรียก check_game_over() เพื่อคำนวณสถานะเกมจบจากกระดานที่โหลดมา แล้วแสดง “Game Loaded!”

make_piece(template_idx, color_idx, slot):

สร้างชิ้นส่วนจากรูปทรงและสีที่กำหนด พร้อมคำนวณตำแหน่งบ้าน (ค่า anchor_x และ anchor_y ของ Piece) ในมือ ตามช่อง slot (0-2) ทั้ง spawn_hand() และ load_game() เรียกใช้ จึงได้ตำแหน่งในมือตรงกัน

1. ดึงรูปทรง จาก SHAPE_TEMPLATES[template_idx][0]
2. หาความกว้างจริงของชิ้นส่วน วนหาค่าคอลัมน์ขวาสุด (max_col) แล้วแปลงเป็นพิกเซล
$$\text{piece_width} = (\text{max_col} + 1) \times 24$$
3. คำนวณตำแหน่ง $\text{new_x} = \text{slot} \times \text{slot_w} + \text{slot_w} / 2 - \text{piece_width} / 2$
เพื่อให้ชิ้นส่วนอยู่กึ่งกลางช่อง และ $\text{new_y} = 490$ คงที่ ใช้หลักการคำนวณเดียวกับ spawn_hand() ของ commit 6

ในเวอร์ชันก่อนหน้านี้ spawn_hand() คำนวณความกว้างของชิ้นส่วนและตำแหน่งกึ่งกลางช่องเอง ภายในฟังก์ชัน ส่วน commit 7 มีแยกการคำนวณนี้ออกมาเป็นฟังก์ชัน make_piece() เพราะ load_game() ต้องสร้างชิ้นส่วนกลับเข้าไปอยู่ในมือที่ตำแหน่งเดิมด้วยวิธีเดียวกัน ถ้าไม่แยกต้องเขียนโค้ดซ้ำสองที่ หากมีข้อผิดพลาดจะทำให้ชิ้นส่วนที่ไหลตมาจะวางผิดตำแหน่ง เวอร์ชันนี้ทั้ง spawn_hand() และ load_game() เรียกใช้ make_piece() จึงได้ตำแหน่งตรงกันเสมอ

4. คืนค่า return Piece(shape, color_idx, new_x, new_y, template_idx)

Format ของไฟล์ savegame.txt

ไฟล์ savegame.txt เป็นไฟล์ข้อความธรรมดา 3 บรรทัด

บรรทัด	ข้อมูลที่เก็บ	ตัวค้น	รูปแบบ
1	ค่าทุกช่องของกระดาน 8 แถว	/ (ใช้ค้นแต่ละแถว) , (ใช้ค้นค่าแต่ละช่อง)	[แถว0]/[แถว1]/... โดย 0 = ช่องว่าง, 1-5 = สีของ block
2	คะแนนและ streak	,	score,combo_streak
3	ชิ้นส่วนในมือ 3 ช่อง	;(ค้นแต่ละช่องในมือ) :(ค้นลำดับรูปทรงและลำดับสี)	ลำดับรูปทรง:ลำดับสี หรือ EMPTY

ตัวอย่างไฟล์

[0,0,0,0,0,0,0,0]/[2,2,0,0,0,0,0,0]/ ... /[0,0,0,0,0,0,0,0]

350,2

4:4;EMPTY;9:4

การแก้ไขเล็กน้อยจาก Commit 6

- Piece.__init__() เพิ่มพารามิเตอร์ template_idx (ค่าเริ่มต้น 0) เก็บเป็น self.template_idx เพื่อจำว่าชิ้นส่วนมาจากรูปทรงลำดับใดใน SHAPE_TEMPLATES ใช้ตอนบันทึกเกม
- spawn_hand() สุ่ม index รูปทรงด้วย random.randint แล้วเรียก make_piece() สร้างชิ้นส่วน และคำนวณตำแหน่งในมือ
- setup() รีเซ็ต combo_streak, status_notice, status_timer, clear_msg และ clear_timer ทุกครั้งที่เริ่มเกมใหม่
- mousePressed() เอา game_over ออกจาก global เพราะอ่านค่าอย่างเดียว และใช้ float(piece.mini_cell) ในการคำนวณ scale